Lab 15 – Backend API Development: Creating RESTful services with AI

Task Descriptor-1: (Setup Fast API Backend) Instructions:

• Install Fas tAPI and Uvicorn.

• Use AI to generate a basic FastAPI application with one endpoint

Sample Code: from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def read_root():

return {"message": "Welcome to

AI-powered FastAPI"}

Expected Output:

Running the server and accessing: http://127.0.0.1:8000/

Response:

{"message": "Welcome to AI-powered FastAPI"}

Task Descriptor-1: Setup FastAPI Backend

1. Prompt for AI Assistant

"Generate a basic FastAPI application that includes installing `fastapi` and `uvicorn`, and sets up a root (`/`) GET endpoint that returns `{"message": "Welcome to AI-powered FastAPI"}`."

```
import subprocess
from fastapi import FastAPI
import nest_asyncio
import uvicorn

# Install FastAPI and Uvicorn
print("Installing FastAPI and Uvicorn...")
```

```python
process = subprocess.run(["pip", "install", "fastapi", "uvicorn"],
capture_output=True, text=True)
print(process.stdout)
if process.stderr:
    print(process.stderr)
print("Installation complete.\n")

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

# For demonstration in Colab, run via nest_asyncio and uvicorn
nest_asyncio.apply()

class Config(uvicorn.Config):
    def __init__(self, app, host="0.0.0.0", port=8000):
        super().__init__(app, host=host, port=port)

config = Config(app)
server = uvicorn.Server(config)
await server.serve() # This line will now execute to start the server
```

```
Installing FastAPI and Uvicorn...
Requirement already satisfied: fastapi in
/usr/local/lib/python3.12/dist-packages (0.135.1)
Requirement already satisfied: uvicorn in
/usr/local/lib/python3.12/dist-packages (0.41.0)
Requirement already satisfied: starlette>=0.46.0 in
/usr/local/lib/python3.12/dist-packages (from fastapi) (0.52.1)
Requirement already satisfied: pydantic>=2.7.0 in
/usr/local/lib/python3.12/dist-packages (from fastapi) (2.12.5)
Requirement already satisfied: typing-extensions>=4.8.0 in
/usr/local/lib/python3.12/dist-packages (from fastapi) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in
/usr/local/lib/python3.12/dist-packages (from fastapi) (0.4.2)
Requirement already satisfied: annotated-doc>=0.0.2 in
/usr/local/lib/python3.12/dist-packages (from fastapi) (0.0.4)
Requirement already satisfied: click>=7.0 in
/usr/local/lib/python3.12/dist-packages (from uvicorn) (8.3.1)
Requirement already satisfied: h11>=0.8 in
/usr/local/lib/python3.12/dist-packages (from uvicorn) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in
/usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.0->fastapi) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in
/usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.0->fastapi)
(2.41.5)
```

```
Requirement already satisfied: anyio<5,>=3.6.2 in
/usr/local/lib/python3.12/dist-packages (from starlette>=0.46.0->fastapi)
(4.12.1)
Requirement already satisfied: idna>=2.8 in
/usr/local/lib/python3.12/dist-packages (from
anyio<5,>=3.6.2->starlette>=0.46.0->fastapi) (3.11)

Installation complete.


INFO:     Started server process [533]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [533]
```

1. Expected Output

```
Running the server and accessing:
http://127.0.0.1:8000/

Response:
{"message": "Welcome to AI-powered FastAPI"}
```

Task Descriptor- 2: (API – Create and Read Items)

Instructions:

• Implement endpoints to add and retrieve products.

• Use a Python list as temporary storage.

Sample Code:

```
items = []

@app.get("/items")

def get_items():
```

return items

@app.post("/items")

def add_item(item: dict):

items.append(item)

return {"message": "Item created",

"item": item}

Expected Output:

GET /items → []

POST /items with {"name":"Pen",

"price":20}

→ {"message":"Item created","item":

{"name":"Pen","price":20}}

Task Descriptor-2: API – Update and Delete Items

1. Prompt for AI Assistant

"Extend the FastAPI application to include endpoints for updating and deleting items. The `/items` list should continue to be used as temporary storage. Implement a `PUT` endpoint at `/items/{item_id}` that takes an `item_id` (integer) and an updated `item` (dictionary), replacing the item at the specified index. Also, implement a `DELETE` endpoint at `/items/{item_id}` that removes the item at the given `item_id` from the list."

```
from fastapi import FastAPI, HTTPException
import nest_asyncio
import uvicorn

app = FastAPI()

# Using a Python list as temporary storage for items
# For simplicity, we'll assume item_id corresponds to the list index.
```

```python
items = []

@app.get("/items")
def get_items():
    return items

@app.post("/items")
def add_item(item: dict):
    items.append(item)
    print(f"Item added: {item}")
    return {"message": "Item created", "item": item}

@app.put("/items/{item_id}")
def update_item(item_id: int, item: dict):
    if 0 <= item_id < len(items):
        items[item_id] = item
        print(f"Item at index {item_id} updated to: {item}")
        return {"message": f"Item {item_id} updated", "item": item}
    raise HTTPException(status_code=404, detail="Item not found")

@app.delete("/items/{item_id}")
def delete_item(item_id: int):
    if 0 <= item_id < len(items):
        deleted_item = items.pop(item_id)
        print(f"Item at index {item_id} deleted: {deleted_item}")
        return {"message": f"Item {item_id} deleted", "item": deleted_item}
    raise HTTPException(status_code=404, detail="Item not found")

# To run this FastAPI application in a Colab environment:
nest_asyncio.apply()

class Config(uvicorn.Config):
    def __init__(self, app, host="0.0.0.0", port=8000):
        super().__init__(app, host=host, port=port)

config = Config(app)
server = uvicorn.Server(config)
await server.serve() # Uncommented to run the server
```

```
INFO:     Started server process [533]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [533]
```

2. Expected Output

Assuming the server is running and `items` contains `[{"name":"Pen","price":20}]` from a previous POST request:

```
PUT /items/0 with {"name":"Notebook","price":15}
→ {"message":"Item 0 updated","item":{"name":"Notebook","price":15}}

GET /items (after PUT) → [{"name":"Notebook","price":15}]

DELETE /items/0
→ {"message":"Item 0 deleted","item":{"name":"Notebook","price":15}}

GET /items (after DELETE) → []
```

Task Descriptor- 3: (Update Existing Item)

Instructions:

• Use AI to generate an endpoint to update an item using its index. Sample Code:

@app.put("/items/{index}")

def update_item(index: int, item: dict):

if index >= len(items):

return {"error": "Item not found"}

items[index] = item

return {"message": "Item updated",

"item": item}

Expected Output:

PUT /items/0 with {"name":"Pencil",

"price":10}

→ {"message":"Item updated","item":

{"name":"Pencil","price":10}}

Task Descriptor-3: Update Item by Index

1. Prompt for AI Assistant

"Generate a FastAPI endpoint to update an existing item in a Python list called `items` using its integer index. The endpoint should be a `PUT` request at `/items/{index}`. If the `index` is out of bounds, return an error message. Otherwise, update the item at that `index` with the provided dictionary `item` and return a success message with the updated item."

```
from fastapi import FastAPI
import nest_asyncio
import uvicorn

app = FastAPI()

# Using a Python list as temporary storage for items
# For simplicity, we'll assume item_id corresponds to the list index.
items = []

# Adding a dummy item for demonstration purposes
items.append({"name": "Existing Item", "price": 100})

@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items) or index < 0:
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}

# To run this FastAPI application in a Colab environment:
nest_asyncio.apply()

class Config(uvicorn.Config):
    def __init__(self, app, host="0.0.0.0", port=8000):
        super().__init__(app, host=host, port=port)

config = Config(app)
server = uvicorn.Server(config)
await server.serve() # Uncomment and execute this to run the server
```

```
INFO:     Started server process [533]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

```
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [533]
```

3. Expected Output

Assuming the server is running and `items` contains at least one item, e.g., `[{"name": "Existing Item", "price": 100}]`:

```
PUT /items/0 with {"name":"Pencil","price":10}
→ {"message":"Item updated","item":{"name":"Pencil","price":10}}

GET /items (after PUT) → [{"name":"Pencil","price":10}]

PUT /items/1 with {"name":"NonExistent","price":5}
→ {"error": "Item not found"}
```

Task Descriptor- 4: (Delete Item)

Instructions:

• Implement deletion of an item by index. Sample Code:

@app.delete("/items/{index}")

def delete_item(index: int):

if index >= len(items):

return {"error": "Item not found"}

removed = items.pop(index)

return {"message": "Item removed",

"item": removed}

Expected Output:

DELETE /items/0

→ {"message":"Item removed","item":

{"name":"Pencil","price":10}}

Task Descriptor-4: Delete Item by Index

1. Prompt for AI Assistant

"Generate a FastAPI endpoint to delete an item from a Python list called `items` using its integer index. The endpoint should be a `DELETE` request at `/items/{index}`. If the `index` is out of bounds, return an error message. Otherwise, remove the item at that `index` from the list and return a success message with the removed item."

```
from fastapi import FastAPI
import nest_asyncio
import uvicorn

app = FastAPI()

# Using a Python list as temporary storage for items
# For simplicity, we'll assume item_id corresponds to the list index.
items = []

# Adding a dummy item for demonstration purposes
items.append({"name": "Pencil", "price": 10}) # Pre-fill for testing deletion

@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items) or index < 0:
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}

# To run this FastAPI application in a Colab environment:
nest_asyncio.apply()

class Config(uvicorn.Config):
    def __init__(self, app, host="0.0.0.0", port=8000):
        super().__init__(app, host=host, port=port)

config = Config(app)
server = uvicorn.Server(config)
await server.serve() # Uncomment and execute this to run the server
```

```
INFO:     Started server process [533]
```

```
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [533]
```

4. Expected Output

Assuming the server is running and `items` *contains* `[{"name":"Pencil","price":10}]`*:*

```
DELETE /items/0
→ {"message":"Item removed","item":{"name":"Pencil","price":10}}

GET /items (after DELETE) → []

DELETE /items/0 (if list is empty or index is out of bounds)
→ {"error": "Item not found"}
```

Task Descriptor -5: (Add Auto-Generated Documentation)

Instructions:

• Use AI to add inline comments and docstrings for all endpoints.

• Optionally, integrate Swagger / Flask-RESTX to auto-generate API documentation.

Sample Code:

@app.route('/items', methods=['GET'])

def get_items():

"""

GET /items

Returns a list of all items in the store.

"""

return jsonify(items)

Expected Output:

• Clear documentation for all endpoints,

e.g., via /docs if using

Swagger.

• Students should be able to see endpoint description, methods, and sample payloads

Task Descriptor-6: Add Auto-Generated Documentation (FastAPI)

1. Prompt for AI Assistant

"Generate a FastAPI application that includes two endpoints: a GET endpoint for `/items` to retrieve all items and a POST endpoint for `/items` to add a new item. Ensure both endpoints have clear docstrings that describe their functionality, expected input, and output. Explain how FastAPI leverages these docstrings to automatically generate interactive API documentation."

```
from fastapi import FastAPI
import nest_asyncio
import uvicorn

app = FastAPI()

# Using a Python list as temporary storage for items
items = []

@app.get(
    "/items",
    summary="Retrieve all items",
    description="Returns a list of all current items in the store."
)
def get_items():
    """
    Retrieve all items.

    This endpoint fetches and returns the entire list of items.
    """
    return items

@app.post(
    "/items",
```

```
    summary="Add a new item",
    description="Adds a new item to the store."
)
def add_item(item: dict):
    """
    Add a new item to the list.

    - **item**: A dictionary representing the item to be added (e.g., `{"name":
"Laptop", "price": 1200}`).

    Returns a success message with the created item.
    """
    items.append(item)
    return {"message": "Item created", "item": item}

# To run this FastAPI application in a Colab environment:
nest_asyncio.apply()

class Config(uvicorn.Config):
    def __init__(self, app, host="0.0.0.0", port=8000):
        super().__init__(app, host=host, port=port)

config = Config(app)
server = uvicorn.Server(config)
await server.serve() # Uncomment and execute this to run the server
```

```
INFO:     Started server process [533]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [533]
```

3. Expected Output

Once the server is running (e.g., via `await server.serve()` in Colab), navigate to `http://127.0.0.1:8000/docs` in your web browser. You should see interactive API documentation (Swagger UI) that includes:

- **Clear documentation for all endpoints:** Each endpoint (`/items` GET and POST) will have its `summary` and `description` displayed.
- **Endpoint methods:** Clearly indicating `GET` and `POST`.

- **Input parameters:** For the POST endpoint, it will show the expected `item` (dictionary) as request body.
- **Sample payloads and responses:** FastAPI automatically infers these from your type hints and docstrings, making it easy for users to understand how to interact with your API.

Example of how the `/items` POST endpoint would appear in Swagger UI:

```
POST /items
Summary: Add a new item
Description: Adds a new item to the store.

Request body:
  item: object
    Example: {"name": "Laptop", "price": 1200}

Responses:
  200 OK:
    description: Successful Response
    content:
      application/json:
        schema:
          type: object
          properties:
            message: {type: string, example: "Item created"}

               item: {type: object, example: {"name": "Laptop",
   "price": 1200}}
```